

CS2310现代操作系统 课程笔记

uu-matter543

2024-2025-2

目录

1	导论	10
1.1	操作系统的功能	10
1.1.1	用户视角	10
1.1.2	系统视角	10
1.1.3	操作系统的定义	10
1.2	计算机系统的组成	10
1.2.1	中断	10
1.2.2	IO 结构	10
1.2.3	存储结构	11
1.3	计算机系统结构	11
1.3.1	单核系统	11
1.3.2	多核系统	11
1.3.3	集群系统	11
1.4	操作系统的执行	11
1.4.1	多程序与多任务	11
1.4.2	双模式与多模式	11
1.4.3	定时器	12
1.5	资源管理	12
1.5.1	进程管理	12
1.5.2	内存管理	12
1.5.3	文件系统	12
1.5.4	外存管理	12
1.5.5	缓存管理	13
1.5.6	IO 管理	13

1.6	安全与保护	13
1.7	虚拟化	13
1.8	分布式系统	13
1.9	内核数据结构	13
1.10	计算环境	13
2	操作系统的结构	14
2.1	操作系统的服务	14
2.2	用户界面	14
2.3	系统调用	14
2.3.1	系统调用接口	14
2.3.2	系统调用的类型	14
2.4	系统服务	15
2.5	链接器与加载器	16
2.6	应用程序特定于操作系统的原因	16
2.7	操作系统的设计与实现	16
2.8	操作系统的结构	16
2.8.1	简单结构	16
2.8.2	分层法	16
2.8.3	微内核	16
2.8.4	模块	17
2.8.5	混合操作系统	17
2.9	操作系统的构建与引导	17
2.9.1	操作系统的生成	17
2.9.2	操作系统的引导	17
2.10	操作系统的调试	17
2.10.1	故障分析	17
2.10.2	性能优化	17
2.10.3	跟踪	17
3	进程	18
3.1	进程的概念	18
3.1.1	进程概述	18
3.1.2	进程状态	18
3.1.3	进程控制块	18
3.1.4	线程	18

3.2	进程调度	19
3.2.1	调度队列	19
3.2.2	上下文切换	20
3.3	进程操作	20
3.3.1	进程创建	20
3.3.2	进程终止	20
3.4	进程间通信	20
3.5	共享内存模型	21
3.6	消息传递模型	22
3.6.1	命名	22
3.6.2	同步	22
3.6.3	缓冲	22
3.7	管道	23
3.8	客户机与服务器的通信	23
3.8.1	套接字	23
3.8.2	远程过程调用	23
4	线程	24
4.1	概述	24
4.1.1	背景	24
4.1.2	优点	24
4.2	多线程编程	24
4.2.1	挑战	24
4.2.2	并行的类型	24
4.3	多线程模型	25
4.3.1	多对一模型	25
4.3.2	一对一模型	25
4.3.3	多对多模型	25
4.4	线程库	25
4.4.1	UNIX 线程	25
4.4.2	Windows 线程	25
4.4.3	Java 线程	25
4.5	隐式线程	26
4.5.1	线程池	26
4.5.2	复刻加入模型	26
4.5.3	OpenMP	26

4.5.4	大中央调度 GCD	26
4.5.5	线程构建模块 TBB	26
4.6	多线程问题	26
4.6.1	系统调用的解释	26
4.6.2	信号处理	27
4.6.3	线程撤销	27
4.6.4	线程本地存储 TLS	27
4.6.5	调度程序激活	27
4.7	操作系统实例	27
4.7.1	Windows	27
4.7.2	Linux	27
5	CPU 调度	28
5.1	基本概念	28
5.1.1	CPU 突发与 IO 突发	28
5.1.2	CPU 调度程序	28
5.1.3	抢占式调度和非抢占式调度	28
5.1.4	分派程序	28
5.2	调度原则	28
5.3	调度算法	28
5.3.1	先到先服务 FCFS	28
5.3.2	最短作业优先 SJF	29
5.3.3	轮转 RR	30
5.3.4	优先级调度	31
5.3.5	多级队列调度	31
5.3.6	多级反馈队列调度	32
5.4	线程竞争	32
5.5	多处理器调度	32
5.5.1	负载平衡问题	32
5.5.2	处理器亲和性	32
5.6	实时 CPU 调度	32
5.7	操作系统实例	32
5.7.1	Linux 调度	32
5.7.2	Windows 调度	33
5.7.3	Solaris 调度	33
5.8	调度算法评估	33

5.8.1	确定性模型	33
5.8.2	排队模型	33
5.8.3	仿真	33
5.8.4	实际应用	33
6	进程同步	34
6.1	同步问题	34
6.2	临界区问题	34
6.3	Peterson 解决方案	34
6.4	硬件同步支持	34
6.4.1	内存屏障	34
6.4.2	硬件指令	35
6.4.3	原子变量	35
6.5	互斥锁	35
6.6	信号量	35
6.6.1	信号量的使用	35
6.6.2	信号量的实现	35
6.7	管程	35
6.8	同步与活性	35
7	同步案例	36
7.1	经典同步问题	36
7.1.1	有界缓冲区问题	36
7.1.2	读者作者问题	36
7.1.3	哲学家就餐问题	37
7.2	内核的同步	38
7.2.1	Windows 同步	38
7.2.2	Linux 同步	38
7.3	POSIX 同步	38
7.3.1	POSIX 互斥锁	38
7.3.2	POSIX 信号量	39
7.3.3	POSIX 条件变量	39
7.4	Java 的同步	39
7.4.1	Java 管程	39
7.4.2	Java 重入锁	39
7.4.3	Java 信号量	39

7.4.4	Java 条件变量	39
7.5	其他方法	39
7.5.1	事务内存	39
7.5.2	OpenMP	39
8	死锁	40
8.1	系统模型	40
8.2	死锁的出现	40
8.3	死锁特点	40
8.3.1	必要条件	40
8.3.2	资源分配图	40
8.4	死锁处理策略	40
8.5	死锁预防	40
8.5.1	取消互斥	40
8.5.2	禁止占有并等待	41
8.5.3	允许抢占	41
8.5.4	禁止循环	41
8.6	死锁避免	41
8.6.1	安全状态	41
8.6.2	资源分配图算法	41
8.6.3	银行家算法	41
8.7	死锁检测的实际应用	43
8.7.1	每种资源只有单个实例	43
8.7.2	每种资源存在多个实例	43
8.7.3	检测算法的使用	43
8.8	死锁恢复	43
9	内存	44
9.1	背景	44
9.1.1	硬件	44
9.1.2	地址绑定	44
9.1.3	虚拟地址与物理地址	44
9.1.4	动态加载与动态链接	44
9.2	连续内存管理	44
9.2.1	动态视角	44
9.2.2	碎片问题	45

9.3	分页内存管理	45
9.3.1	基本方法	45
9.3.2	硬件支持	45
9.3.3	保护	45
9.3.4	共享页	45
9.4	页表结构	45
9.4.1	多级页表	45
9.4.2	哈希页表	46
9.4.3	倒置页表	46
9.4.4	TLB 的组织	46
9.5	交换	46
9.6	实际应用	46
9.6.1	x86 架构	46
9.6.2	x64 架构	46
9.6.3	ARM-v8 架构	46
10	虚拟内存	47
10.1	背景	47
10.2	请求调页	47
10.2.1	基本概念	47
10.2.2	空闲帧列表	47
10.2.3	请求调页的性能	47
10.3	写时复制	48
10.4	页面置换算法	48
10.4.1	基本页面置换	48
10.4.2	FIFO 页面置换	48
10.4.3	LRU 页面置换	48
10.4.4	OPT 页面置换	48
10.4.5	LRU 算法的实现与近似	49
10.5	帧分配	50
10.5.1	帧的最小值与最大值	50
10.5.2	全局分配与局部分配	50
10.5.3	非均匀内存访问	50
10.6	抖动	50
10.6.1	抖动现象	50
10.6.2	抖动预防	50

10.7 内存压缩	50
10.8 虚拟内存性能的其他因素	51
10.9 操作系统实例	51
10.9.1 Linux	51
10.9.2 Windows	51
10.9.3 Solaris	51
11 大容量存储	52
11.1 大容量存储结构	52
11.1.1 硬盘驱动器 HDD	52
11.1.2 非易失性存储设备 NVM	52
11.1.3 地址映射	52
11.2 HDD 调度	52
11.2.1 FCFS 调度	52
11.2.2 SCAN 调度	52
11.2.3 C-SCAN 调度	52
11.2.4 算法选择	53
11.3 NVM 调度	53
11.4 错误检测和纠正	53
11.5 存储设备管理	53
11.6 交换空间管理	53
11.7 存储连接方式	53
12 IO 系统	54
12.1 概述	54
12.2 硬件	54
12.2.1 内存映射 IO	54
12.2.2 中断	54
12.2.3 直接内存访问 DMA	54
12.3 应用程序的 IO 接口	54
12.4 内核 IO 子系统	55
12.5 IO 请求与设备操作转换	55
12.6 流	55
12.7 性能提高	55
13 文件系统接口	56
13.1 文件	56

13.1.1 文件属性	56
13.1.2 文件操作	56
13.1.3 文件类型	56
13.1.4 文件结构	56
13.2 访问方法	56
13.3 目录	56
13.4 保护	56
14 文件系统实现	57
14.1 文件系统结构	57
14.2 文件系统操作	57
14.3 目录实现	57
14.4 分配方法	57
14.5 空闲空间管理	57
15 文件系统内部细节	58
15.1 存储系统	58
15.2 文件系统挂载	58
15.3 分区与挂载	58
15.4 文件共享	58
15.5 虚拟文件系统	58
15.6 远程文件系统	58
15.7 一致性语义	58

1 导论

1.1 操作系统的功能

从用户视角俯视，计算机系统可分为用户视角、应用程序、操作系统、硬件 4 层

1.1.1 用户视角

用户关注的是计算机系统的易用性和性能，需要操作系统实现硬件资源的充分利用

1.1.2 系统视角

操作系统的作用：为应用程序分配资源；控制程序，防止系统错误和硬件的不当使用

1.1.3 操作系统的定义

操作系统是特殊的计算机软件，由**内核**和**系统程序**组成，其中内核是不停运行的；应用程序在操作系统上运行

1.2 计算机系统的组成

1.2.1 中断

中断是处理系统错误，或设备请求 CPU 执行特定功能的方式；操作系统就是中断驱动的，其中硬件中断过程如下：

1. CPU 在执行完每条指令之后都要检查是否有中断请求
2. 设备驱动程序通过硬件发送中断信号到 CPU
3. CPU 检测到中断请求后，保存被中断程序的信息
4. CPU 通过中断向量表，查找中断处理程序的入口，并执行中断处理程序
5. 中断处理程序结束，归还操作权，CPU 恢复被中断的程序

中断也可由软件直接发起，可能的原因是出现错误（除零、无效寻址、无限循环、更改操作系统重要部件）或等待用户进一步指令

操作系统的启动程序也是中断处理程序，存放于只读内存（也称为固件或 ROM）中，在每次开机时自动执行，作用是启动其它硬件设备，以及将操作系统内核载入内存并执行

1.2.2 IO 结构

- 非抢占式：IO 操作完成时控制权才返回程序
- 抢占式：程序不等待 IO 操作完成即夺回控制权，除非程序发送等待 IO 完成的请求

1.2.3 存储结构

内存：程序指令存放的位置，CPU 从中读取指令并执行

- 支持随机访问（可根据地址访问内存任意位置）
- 一般是易失的（断电后内存中的数据丢失）

二级存储：提供大容量、非易失的存储，但访问速度较慢，主要分为 HDD 和 NVM，当前 HDD 较为落后，NVM 占主导地位

存储的层级结构：考虑到速度、成本、易失性，将存储设计为层级结构，从内向外依次为寄存器、缓存、内存、二级存储 (NVM,HDD)、三级存储 (光盘,磁带)，访问速度依次降低，容量依次增大；一级存储易失（断电后数据丢失），二级存储和三级存储非易失（断电后数据保留）；层级结构设计的目的是根据 CPU 访问的频数，将外部存储中的内容依次前递，以加快 CPU 存取数据的速度

1.3 计算机系统结构

1.3.1 单核系统

单核系统的 CPU 只含 1 个处理器核，多用于特殊用途和嵌入式系统（例如家电控制）

1.3.2 多核系统

- 不对称多处理：每个处理器处理特定任务
- 对称多处理：每个处理器能处理所有任务

1.3.3 集群系统

通过内网将多台计算机组合在一起，用于提供高可用性服务和高性能计算

1.4 操作系统的执行

计算机的启动过程：加载内核到内存；启动内核以外的服务；等待中断事件发生

1.4.1 多程序与多任务

- 多程序：内存中保存多个进程，当某个进程需要等待时 CPU 可切换到其它进程
- 由操作系统中的 CPU 调度组件完成任务切换

1.4.2 双模式与多模式

- 双模式：用户模式与内核模式，通常由硬件区分
- 内核模式用于保护操作系统本身和硬件系统

- 多模式：多应用于虚拟化技术，例如虚拟机管理模式

只能在内核模式运行的指令称为特权指令；如果应用需要执行特权指令，则发起系统调用，操作系统接管控制权后切换为内核模式执行指令，之后返回结果并切换回用户模式

1.4.3 定时器

- 作用：防止死循环，防止应用程序长时间占用 CPU 导致操作系统无法介入
- 定时器通过硬件实现，操作系统通过特权指令设置定时器

1.5 资源管理

1.5.1 进程管理

进程与线程概述：

- 程序与进程的区别：程序是静态的，进程是动态的
 - 进程需要硬件资源（内存，CPU，IO，外存中的文件）以运行
 - 进程结束需要将资源及时回收
 - 线程：允许单个进程拥有多个线程，由多个处理器核执行进程的指令
 - 进程分为系统进程和用户进程，可在单 CPU 上交替执行，也可在多 CPU 上并行执行
- 操作系统的进程管理作用：创建和删除进程，暂停和恢复进程，提供进程同步机制，提供进程间通信，调度进程和线程，防止死锁出现

1.5.2 内存管理

内存概述：程序的运行需要指令和数据处于内存中

操作系统的内存管理作用：记录内存的使用情况，包括是否在使用、使用者；决定进程在内存与交换区的调入调出；根据需要分配和回收内存

1.5.3 文件系统

文件系统的目标：为存储设备提供逻辑视图，将物理设备抽象为逻辑存储单元

文件管理的目的：将文件系统映射为目录，决定程序对文件的操作权限

操作系统的文件管理作用：创建和删除文件，提供用户对文件的操作接口，备份文件

1.5.4 外存管理

外部存储：存放内存中放不下的、需要长时间保存的数据

操作系统的外存管理作用：安装和卸载外存，管理外存空闲空间，为目标分配空间，外存访问的调度，保护外存内容

1.5.5 缓存管理

缓存：访问速度仅次于寄存器的存储，操作系统管理 CPU 需要的数据

1.5.6 IO 管理

操作系统的 IO 管理作用：隐藏具体硬件设备的特性，将其抽象为通用接口

1.6 安全与保护

安全与保护：限制用户进程访问资源，保护计算机硬件和操作系统内核不被攻击

操作系统的安全与保护作用：为不同用户分配不同权限，以限制该用户的进程的行为

1.7 虚拟化

虚拟化技术：将计算机硬件拆分，每个部分能够运行不同的操作系统

虚拟化的应用：在单个操作系统上开发多个操作系统的程序，运行不同的计算环境

1.8 分布式系统

分布式系统：不同组件物理上分开、可能异构、通过内网相连的计算机系统

需要分布式操作系统提供文件共享、信息交换等功能

1.9 内核数据结构

线性结构：线性表：顺序表，链接表；堆栈；队列等

树状结构：普通树、二叉树、查找树等

哈希表：将查找数据作为输入，运算后得到输出，以其为索引快速访问到目标数据

位图：用二进制比特组成的串表示 n 个物品的状态

1.10 计算环境

传统计算环境：包括网络服务器、网络终端、无线网络等

移动计算：类似智能手机和平板电脑的便携式、可移动计算

服务器客户机模型：客户端提出需求，由服务端进行处理

P2P 对等计算：分布式系统的另一种形式，每个结点可同时作为服务器和客户机

云计算技术：通过网络提供计算机系统相关的服务，多基于虚拟化技术

实时嵌入式系统：操作系统必须在某时间内完成某项任务（时间敏感型任务）

2 操作系统的结构

2.1 操作系统的服务

总述：操作系统提供应用程序在计算机硬件上运行的环境

- 用户界面：包括图形界面、命令行界面、触摸屏界面，用于计算机与用户交互
- 程序执行：将应用程序加载到内存并运行，在运行结束后结束程序
- 输入输出：涉及外存文件或输入输出设备
- 文件系统：管理程序进行文件操作的权限
- 通信：进程间通信、不同计算机系统的进程通信
- 错误检测：检测并修复可能出现的错误
- 资源管理：为进程分配 CPU 运行周期、内存、外存、IO 等资源
- 日志：记录程序应用资源的情况
- 保护与安全：控制进程不侵害其它进程和计算机系统

2.2 用户界面

命令行界面：由内核或系统程序 Shell 实现，用于获取并执行用户指令

图形用户界面：显示器上的图标代表程序、文件、操作等，通过鼠标点击互动

触摸屏界面：多应用于不能支持鼠标的移动设备，主要互动方式是在触摸屏上做手势

2.3 系统调用

提供操作系统服务的封装接口 API，这样应用程序的开发人员无需知道内部实现细节，只需知道调用方式就可使用

2.3.1 系统调用接口

操作系统能够提供的系统调用通常存放在特定向量中，这样只要给出索引就可以找到对应系统调用的执行入口

系统调用在内核中执行，返回状态参量和系统调用结果

系统调用往往需要参数，参数传递通常有以下形式：

- 通过寄存器传递（参数较多时不适用）
- 通过内存传递，同时向寄存器传递参数地址
- 将参数压入堆栈，系统调用过程中操作系统自动加载堆栈中的参数

2.3.2 系统调用的类型

- 进程控制：

- 创建进程，删除进程，结束进程，中止进程
- 加载程序到内存，执行内存中的程序
- 获取和修改进程的属性参数
- 等待 CPU 执行周期或事件发生
- 分配内存，释放内存，内存转储（处理错误）
- 断点执行（用于 debug 机制）
- 同步锁机制等
- 文件管理：
 - 创建文件，删除文件，打开文件，关闭文件
 - 读写文件，重定位文件，移动文件，复制文件
 - 获取和修改文件的属性参数
- 设备管理：
 - 请求设备，归还设备
 - 读写设备内容，重定位设备
 - 获取和修改设备的属性参数
 - 逻辑上处理设备的添加和移除
- 信息维护：
 - 获取和修改时间或日期
 - 维护其它系统参数
 - 维护进程、文件、设备的属性
- 通信：创建和删除通信连接，发送和接受信息，添加和移除远程设备
- 保护：控制计算机资源的访问权，维护权限信息

2.4 系统服务

目的：为应用程序开发和执行提供便利环境

- 文件管理类：创建、删除、复制、重命名、打印、转储、列出文件，查找和修改文件内容，访问和操作文件目录等
- 状态信息类：获取日期、时间、存储空间、用户信息、设备参数、日志、调试信息、注册表（操作系统整合配置信息后的存储格式）等
操作系统负责获取这些信息并格式化，以供应用程序使用
- 程序语言支持：编译器、汇编器、调试器、解释器等
- 程序加载执行：绝对地址加载程序、重定位地址加载程序、机器语言和高级语言的调试程序等
- 通信：提供进程之间通信、计算机之间通信的机制
- 后台服务：硬盘检查、进程调度、错误检测等功能

2.5 链接器与加载器

- 可重定位目标文件：可被加载到内存的任何物理地址的目标文件
- 加载器：将程序载入内存并执行的操作系统组成部分
载入内存过程中，**重定位**为程序分配物理地址，并调整程序中的地址以匹配实际地址
- 动态链接库：将库按需加载至内存，可由进程间共享

过程总览：源程序经过编译器，编译为目标程序；再经过链接器与其他目标文件链接，成为目标程序；最后经过加载器加载为进程，并结合动态链接库共同完成执行

2.6 应用程序特定于操作系统的原因

根本原因：不同操作系统有不同的系统调用和文件系统

解决方式：用可解释的语言编写（该语言具备多系统的解释器）；用虚拟机进行开发；用标准语言编写，并在不同操作系统上编译

2.7 操作系统的设计与实现

设计操作系统时，既要考虑到底层硬件，又要让用户便于使用、易于学习、可靠、安全、快速，还要考虑到系统本身的灵活、稳定、正确、高效，便于设计、实现和维护

机制与策略分离：机制决定如何做，策略决定做什么；机制和策略分离保证了灵活性

早期操作系统由汇编语言完成，现代操作系统由高级语言完成，由多种语言混合而成，具有可移植性，可通过仿真在各种硬件上运行

2.8 操作系统的结构

2.8.1 简单结构

操作系统由系统程序和内核组成，内核是硬件以上、系统调用接口以下的所有内容

2.8.2 分层法

将操作系统分为若干层级，最内层为硬件，最外层为用户操作接口

内层对外层封装为接口，提供函数和服务等

2.8.3 微内核

将内核模块化为组件，并将不必要放入内核的组件移出内核，优点：

- 易于扩展微内核本身
- 易于扩展操作系统，适应不同的计算机体系结构
- 运行在内核模式的代码更少，更稳定，更安全

2.8.4 模块

运用面向对象方法，将每个模块独立，模块间交流通过接口，并按需加载内核模块

2.8.5 混合操作系统

将不同操作系统组合形成混合系统，解决可用、性能、安全等问题

2.9 操作系统的构建与引导

2.9.1 操作系统的生成

操作系统的生成步骤：

1. 编写操作系统代码
2. 根据要运行的硬件系统调整
3. 编译操作系统、导入操作系统
4. 启动计算机及操作系统

2.9.2 操作系统的引导

操作系统的引导步骤：

1. 启动时，硬件固定读取位于只读内存的特定位置的代码，从而定位内核
2. 内核加载到内存并运行，过程中初始化其它硬件并挂载文件系统

引导程序称为 BIOS，现代操作系统大多为 UEFI；此外，引导管理器 GRUB 允许选择不同的内核版本和操作系统

2.10 操作系统的调试

2.10.1 故障分析

操作系统运行时，应当生成日志文件，用于查看错误信息

应用程序或内核崩溃时，应当对内存进行转储，保存状态信息以供调试参考

2.10.2 性能优化

操作系统中应当包含计数器，记录进程的活动信息用于性能分析

2.10.3 跟踪

操作系统应当提供工具，用于跟踪进程的系统调用等活动

3 进程

3.1 进程的概念

进程就是正在内存中运行的程序

3.1.1 进程概述

- 进程的内存映像：
 - 可执行的机器代码（位于文本段）
 - 目前的活动（程序计数器、寄存器等）
 - 栈段（函数调用的临时数据存储，向下生长）
 - 数据段（存放全局变量）
 - 堆段（程序运行的动态内存，向上生长）
- 进程与程序：
 - 程序是存储于外存的可执行文件，载入内存后成为进程
 - 可通过执行命令或者鼠标点击以基于程序创建进程
 - 单个程序可能生成多个进程

3.1.2 进程状态

进程状态指进程当前的活动，与执行的过程有关，包括：

- 新建：将程序载入内存，创建进程
- 就绪：进程等待分配处理器
- 等待：进程等待 IO 操作或事件发生
- 运行：程序指令正在执行
- 终止：进程已完成执行或被强制杀死

进程状态间的迁移过程如图1所示

3.1.3 进程控制块

进程在操作系统中用进程控制块 PCB 表示，存储进程的内存映像，以及用于 CPU 调度、内存调度、IO 分配的信息

3.1.4 线程

多线程：单个程序创建多个线程，拥有多个程序计数器，同时执行多个地址上的指令，需要进程控制块包含多线程的信息

在 Linux 系统中，进程用 `<linux/sched.h>` 库中的 `task_struct` 结构体表示

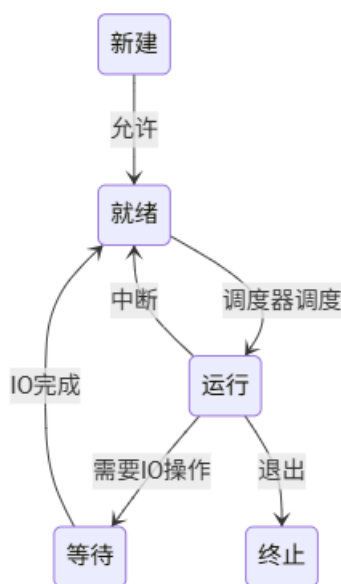


图 1: 进程状态 FSM

3.2 进程调度

进程调度就是调度器选择在 CPU 上执行的进程

3.2.1 调度队列

调度队列是存放同状态进程的 PCB 的数据结构，通常用链接队列作为容器，包括就绪队列和各种等待队列，进程调度的实质就是进程在不同队列间迁移，如图2所示

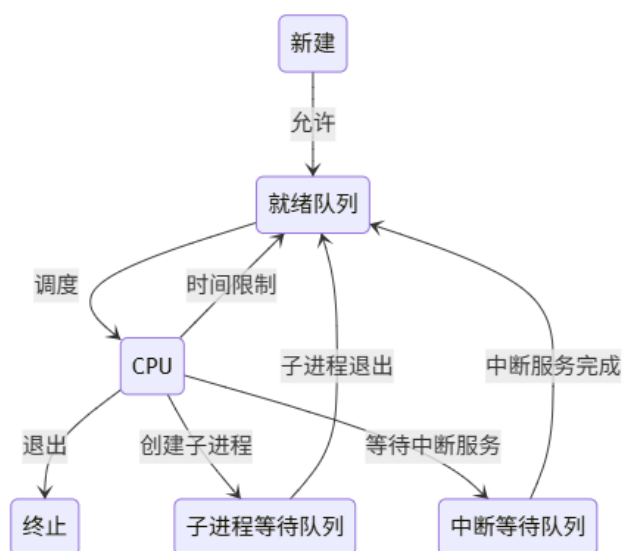


图 2: 进程调度示意图

3.2.2 上下文切换

上下文切换，即 CPU 从一个进程切换到另一个进程，过程如下：

1. 保存当前进程 P0 的 PCB0，读取其它进程 P1 的 PCB1
2. P1 结束或中断时，释放或保存 PCB1，加载 PCB0 或者其它进程的 PCB

代价问题：上下文切换的时间不是系统完成任务的必需工作，是额外的开销；操作系统和 PCB 越复杂，内存管理系统越复杂，上下文切换的开销越大

3.3 进程操作

3.3.1 进程创建

在操作系统中，每个进程都有唯一的进程标识符 PID

父进程创建子进程，子进程再创建其它进程，可形成进程树

资源共享模型：共享所有资源，或共享部分资源，或资源相互独立

运行顺序模型：并行运行，或父进程等待子进程退出

UNIX 系统中的进程创建函数：

- `fork()` 创建子进程
- `exec()` 用其它程序代替当前进程剩余的代码执行
- `wait()` 父进程等待子进程结束，并获取子进程的退出状态

具体用法如图3所示

3.3.2 进程终止

- `exit()` 进程正常退出，请求操作系统释放其空间
- `abort()` 父进程强制子进程退出，原因可能是：
 - 父进程不再需要子进程的任务
 - 父进程正在退出，子进程应一并退出
 - 子进程使用了超出为子进程分配的资源
- 级联终止：部分操作系统不允许无父进程的子进程存在，父进程退出时，由操作系统启动级联终止，强制其子进程全部退出
- 僵尸进程：子进程已终止，但父进程未调用 `wait()` 获取退出状态，导致子进程的最终状态持续在内存中不能被释放
- 孤儿进程：其父进程未调用 `wait()` 而父进程已终止

3.4 进程间通信

进程间可以独立，也可以协作，协作的进程间可互相影响，要求进程间能够通信

进程间通信的好处：信息共享，加速计算，模块化

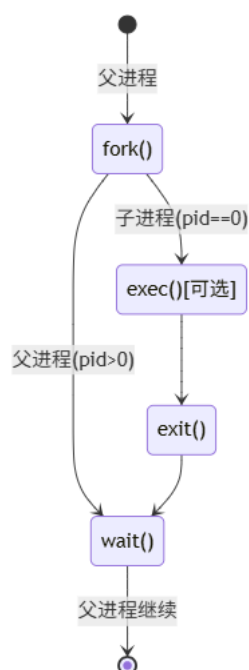


图 3: 子进程创建

进程间通信的机制：共享内存和消息传递

多进程实例：Chrome 内核浏览器

- 浏览器进程：管理用户界面和磁盘 IO
- 渲染器进程：处理 HTML 和 JavaScript 等，渲染页面
- 插件进程：扩展浏览器功能，管理浏览器进程间通信

生产者消费者模型：生产者生产信息，消费者消费信息，通过缓冲区交流

- 无界缓冲区：缓冲区大小无限（理想化模型）
- 有界缓冲区：缓冲区空时消费者等待；缓冲区满时生产者等待

3.5 共享内存模型

多进程通过内存空间共享信息，由用户进程而非操作系统控制

有界缓冲区的实现：创建大小为 `buffer_size-1` 的循环队列如下

```
struct item {...};
struct item buffer[buffer_size];
int head = 0, tail = 0;
```

生产者进程：将信息对象 `next_produced` 存入缓冲区如下

```
struct item next_produced;
while(true)
```

```
{  
    while ((tail+1) % buffer_size == 0); //wait while buffer is full  
    buffer[tail] = next_produced;  
    tail = (tail+1) % buffer_size;  
}
```

消费者进程：从缓冲区取得信息对象 next_consumed 如下

```
struct item next_consumed;  
while(true)  
{  
    while (head==tail); //wait while buffer is empty  
    next_consumed = buffer[head];  
    head = (head+1) % buffer_size;  
}
```

3.6 消息传递模型

消息传递模型允许进程间交流并同步，由内核管理，进程通信前需要建立链接，之后通过 send 和 message 操作进行交换信息

3.6.1 命名

直接通信：每个 send 和 message 操作都由发起进程指定目标对象

链路特点：自动建立链路，每两个进程与特定链路一一对应，一般为双向

间接通信：通过邮箱进行通信，每个邮箱都有唯一的标识符，拥有共享邮箱的进程间才可通信；创建邮箱后，每个 send 和 message 操作都由发起进程指定目标邮箱

链路特点：一个链路可与若干进程关联，两个进程间可有多条链路，可以单向或双向

问题：多进程共享邮箱中，单个进程进行 send 操作时多个进程正在等待 receive 操作

解决：只允许一个邮箱关联两个进程，退化为直接通信；将进行 receive 操作的进程排列为队列依次处理；随机选择接收方；由发送方指定接收方

3.6.2 同步

同步，即阻塞，是指进行操作的一方在己方或对方接收成功时才恢复其它任务

异步，即非阻塞，是指进行操作的一方在操作后立即切换到其他任务

3.6.3 缓冲

与共享内存类似，通信链路也可设置容量

- 零缓冲区：缓冲区无容量，发送方总是阻塞，等待消息的接收
- 有界缓冲区：缓冲区空时接收方等待；缓冲区满时发送方等待
- 无界缓冲区：缓冲区大小无限（理想化模型），发送方总是无需等待

3.7 管道

管道是两个进程间通信的媒介，支持大量信息的传输

匿名管道

- 按照生产者消费者模型进行通信
- 管道是单向的，只允许单向通信
- 需要进程间有父子关系

命名管道

- 无需进程间有父子关系
- 支持多个进程通信
- 支持双向通信

3.8 客户机与服务器的通信

3.8.1 套接字

套接字由 IP 地址和端口号组成，IP 地址用于区分服务器，端口号用于区分请求的内容；通信在匹配成对的套接字之间实现，分为 TCP 套接字、UDP 套接字、广播套接字等

3.8.2 远程过程调用

将通过网络系统的过程调用进行抽象，用端口号区分服务类型，过程如下：

1. 客户机调用 RPC 请求到服务器进程 X
2. 客户机内核发送信息，用于查找进程 X 的端口号
3. 服务器收到消息，回复进程 X 的端口号 P 并开放端口 P
4. 客户机收到端口号 P，将 P 与 RPC 参数封装并发送
5. 服务器上监听端口 P 的进程收到消息，对应进程处理参数并输出数据
6. 客户机内核收到服务器响应，返回给用户

4 线程

4.1 概述

现代应用程序以多线程为主，线程在进程内运行

4.1.1 背景

多任务：应用程序的多个任务可分发给多个线程完成

开销与代价：线程创建的开销远小于进程创建，且多线程相对容易编码和开发
操作系统本身作为不断运行的进程，也是多线程的

4.1.2 优点

- 响应性好：即使进程的某些部分被阻塞，未被阻塞的部分仍可正常执行
- 资源共享：线程间可共享同一进程的资源，相比于进程间通信更为简单
- 开销较小：线程的创建和切换代价均小于进程的创建和切换
- 可伸缩性：同一进程多个线程能充分利用多核 CPU 架构的优势

4.2 多线程编程

4.2.1 挑战

多线程编程主要面临的问题有：任务识别与接取、平衡、数据分割、数据依赖、调试
并行与并发的区别：

- 并行指同时执行多个任务，必须依赖多处理器
- 并发指一段时间内每个任务都有进展，而在特定时刻可能只有单个任务执行；不依赖多处理器，单核系统可由调度器提供并发功能

4.2.2 并行的类型

- 数据级并行：将数据分发给多个处理器核，每个处理器核执行相同操作
 - 任务级并行：将线程分发给多个处理器核，每个处理器核执行不同任务
- 阿姆达尔定律：加速比公式

$$Speedup = \frac{1}{B + \frac{1-B}{n}}$$

其中 B 为必须串行部分占比， n 为并行线程数

4.3 多线程模型

4.3.1 多对一模型

将多个用户线程映射到单个内核线程，兼容单核系统，但是单线程阻塞会导致整个进程阻塞，而且不能发挥多核系统的优势

4.3.2 一对一模型

将单个用户线程映射到单个内核线程，具有更好的并发性，但是会产生大量内核线程，导致系统负担增大

4.3.3 多对多模型

将多个用户线程灵活映射到多个内核线程，这样当某个用户线程阻塞导致对应内核线程不可用时，其它用户线程可使用另一个内核线程；此外，多对多模型还兼容多对一模型和一对一模型的存在

4.4 线程库

线程库是操作系统为程序员提供创建和管理线程的 API 集合

4.4.1 UNIX 线程

UNIX 系统中可使用系统库 `pthread.h` 管理线程，但只是规范线程行为，而非定义线程的工作；主进程可用 `pthread_join` 函数等待子线程结束

4.4.2 Windows 线程

Windows 系统中可使用系统库 `windows.h` 管理线程，同理也只是规范线程行为；主进程可用 `WaitForMultipleObjects` 函数等待子线程结束，还可设置等待时长、等待有线程返回或等待全部线程返回等

4.4.3 Java 线程

Java 线程由 Java 虚拟机 JVM 管理，可在任何提供了 JVM 的操作系统上运行；可从 `Thread` 类派生并重载 `run` 函数创建新线程；主进程可用 `join` 函数等待子线程结束

4.5 隐式线程

4.5.1 线程池

在池子中预先创建等待工作的线程，优点是速度更快、限制总线程数量防止死锁

4.5.2 复刻加入模型

主进程将任务分发给分支线程后，可用 fork-join 机制等待子进程结束，算法模型如下：

```
def task(n):
    if n < threshold:
        solve(n)
    else:
        subtask_1 = fork(new task(o(n)))
        subtask_2 = fork(new task(o(n)))
        ...
        result_1 = join(subtask_1)
        result_2 = join(subtask_2)
        ...
    return combine_results(result_1, result_2, ...)
```

4.5.3 OpenMP

OpenMP 是支持 C 和 C++ 语言的多线程编程库，包含预编译指令和 API 等
使用 `#pragma omp parallel` 指令会创建多个线程，每个线程同时执行语句块的内容

4.5.4 大中央调度 GCD

GCD 是 MacOS 和 iOS 采用的一项技术，包括运行时动态库、语言扩展和 API，允许创建并行语句块，且可调度语句块在线程池中的线程上执行

4.5.5 线程构建模块 TBB

TBB 是 Intel 发行的 C 语言并行编程工具库

4.6 多线程问题

4.6.1 系统调用的解释

`fork()` 调用可能复制进程的所有线程，也可能只复制调用了 `fork()` 的线程，与系统有关；`exec()` 调用的程序会替代所有线程

4.6.2 信号处理

信号用于通知进程某个事件的发生，由内核运行的信号处理程序进行处理（该程序可以是默认的，也可以是用户自定义重载的），通知给信号适用的所有线程，或某一进程的所有线程，或某一进程的部分线程，或进程指定的专门用于处理信号的线程

4.6.3 线程撤销

线程撤销，指在线程退出之前终止线程，分为异步撤销（由其它线程立刻终止目标线程）和延迟撤销（线程每运行一段就检查自己是否应当终止）

Pthread 中可用 `pthread_cancel()` 函数终止目标线程；可设置线程的三种撤销模式：不能撤销、延迟撤销、异步撤销

4.6.4 线程本地存储 TLS

线程本地存储允许线程拥有本地数据，可在不启动进程时启动线程

4.6.5 调度程序激活

用户模式与内核模式之间存在轻量级进程 (LWP)，用于用户线程和内核线程的通信

4.7 操作系统实例

4.7.1 Windows

Windows 系统使用一对一模型，线程的上下文包括唯一的线程标识符、寄存器组状态、用户模式堆栈和内核模式堆栈、运行时库和动态链接库、程序计数器

线程的数据结构：

- **ETHREAD** 执行线程块，位于内核，包括指向所属进程的指针、指向 **KTHREAD** 的指针
- **KTHREAD** 内核线程块，位于内核，包含调度和同步信息、内核堆栈、指向 **TEB** 的指针
- **TEB** 线程环境块，位于用户空间，包含线程标识符、用户态堆栈、逻辑存储信息

4.7.2 Linux

Linux 系统不区分进程和线程，使用 **task** 统称任务

- **clone()** 系统调用：创建与当前任务共享资源的新任务，类似于创建线程
- **fork()** 系统调用：创建与当前任务不共享资源的新任务，类似于创建进程

5 CPU 调度

5.1 基本概念

5.1.1 CPU 突发与 IO 突发

进程执行的过程实质上是 CPU 突发和 IO 突发的交替
突发具有长尾效应，即包括大量的短时突发和少量的长时突发

5.1.2 CPU 调度程序

CPU 调度程序的任务是：根据调度算法，在等待队列中选择进程在 CPU 上执行

5.1.3 抢占式调度和非抢占式调度

进程状态迁移与调度：

- 存在进程由运行状态切换为就绪状态的调度，为抢占式调度
- 不存在上述切换的调度，为非抢占式调度

抢占式调度的问题：同步问题、任务重要性问题

5.1.4 分派程序

分派程序是收到 CPU 调度程序的指示时进行 CPU 控制权转移的程序，其任务包括上下文切换、内核态切换到用户态、重新启动用户程序等

5.2 调度原则

调度原则是提高 CPU 利用率、提高单位时间内完成的进程数量最大，评估指标如下：

- 周转时间：进程从提交到完成的时间
- 等待时间：进程从提交到完成的过程中等待的时间（i.e. 周转时间减去执行时间）
- 响应时间：进程从提交到首次执行的时间

5.3 调度算法

5.3.1 先到先服务 FCFS

先到达的进程先得到服务，后到达的进程后得到服务

优点：算法简单易理解；缺点：平均等待时间较长

例 设突发时长 $(P_1, P_2, P_3) = (24, 3, 3)$ ，分别求 FCFS 下，按照 $\langle P_1, P_2, P_3 \rangle$ 顺序和 $\langle P_3, P_2, P_1 \rangle$ 顺序到达的平均周转时间、平均等待时间、平均响应时间

解 $\langle P_1, P_2, P_3 \rangle$ 顺序下：0–24 处理 P_1 ，24–27 处理 P_2 ，27–30 处理 P_3

进程	等待时间	响应时间	周转时间
P_1	0	0	24
P_2	24	24	27
P_3	27	27	30

平均等待时间为 17，平均响应时间为 17，平均周转时间为 27

$\langle P_3, P_2, P_1 \rangle$ 顺序下：0–3 处理 P_3 ，3–6 处理 P_2 ，6–30 处理 P_1

进程	等待时间	响应时间	周转时间
P_1	6	6	30
P_2	0	0	3
P_3	3	3	6

平均等待时间为 3，平均响应时间为 3，平均周转时间为 13

大量进程等待突发时间较长的进程退出称为护航效应，会降低吞吐量

5.3.2 最短作业优先 SJF

选择 CPU 突发时间最短的进程交由 CPU 执行

优点：具有最短的平均等待时间；缺点：实际应用中难以预知突发时间

例 设突发时长 $(P_1, P_2, P_3, P_4) = (6, 8, 7, 3)$ ，求 SJF 下的平均周转时间、平均等待时间、平均响应时间

解 0–3 处理 P_4 ，3–9 处理 P_1 ，9–16 处理 P_3 ，16–24 处理 P_2

进程	等待时间	响应时间	周转时间
P_1	3	3	9
P_2	16	16	24
P_3	9	9	16
P_4	0	0	3

平均等待时间为 7，平均响应时间为 7，平均周转时间为 13

当进程按照特定序列到达时，最短作业优先调度可描述为：将控制权交给当前时刻剩余突发时间最短的进程

例 设突发时长 $(P_1, P_2, P_3, P_4) = (8, 4, 9, 5)$ ，到达时间 $(P_1, P_2, P_3, P_4) = (0, 1, 2, 3)$ ，求 SJF 下的平均周转时间、平均等待时间、平均响应时间

解 0-1 处理 P_1 ，1 时 P_2 到达，剩余时间 $(P_1, P_2) = (7, 4)$ ，控制权移交 P_2 ，1-2 处理 P_2 ，2 时 P_3 到达，剩余时间 $(P_1, P_2, P_3) = (7, 3, 9)$ ，控制权不变，2-3 处理 P_2 ，3 时 P_4 到达，剩余时间 $(P_1, P_2, P_3, P_4) = (7, 2, 9, 5)$ ，控制权不变，3-5 处理 P_2 ，5-10 处理 P_4 ，10-17 处理 P_1 ，17-26 处理 P_4

进程	等待时间	响应时间	周转时间
P_1	9	0	17
P_2	0	0	4
P_3	15	15	24
P_4	2	2	7

平均等待时间为 6.5，平均响应时间为 4.25，平均周转时间为 13

5.3.3 轮转 RR

存在其它进程时，每个进程只能在连续执行特定时间，耗尽时间限制后则会切换进程
轮转调度：进程耗尽时间限制后，会被放到等待队列的末尾，并切换下个进程
算法性能很大程度上取决于时间配额的大小

- 过大会与 FCFS 类似
- 过小会频繁进行进程切换，开销很大

较合理的是 80% 的进程 CPU 突发时间小于时间限制

例 设突发时长 $(P_1, P_2, P_3) = (24, 3, 3)$ ，时间限制为 4，求 RR 下的平均周转时间、平均等待时间、平均响应时间

解 0-4 处理 P_1 ，4-7 处理 P_2 ，7-10 处理 P_3 ，10-30 处理 P_1

进程	等待时间	响应时间	周转时间
P_1	6	0	30
P_2	4	4	7
P_3	7	7	10

平均等待时间为 5.67，平均响应时间为 3.67，平均周转时间为 15.67

5.3.4 优先级调度

优先级调度中，每个进程都关联数字表示优先级，CPU 会分配给优先级最高的进程
无穷阻塞/饥饿问题：高优先级进程持续涌入导致低优先级进程得不到服务

例 设突发时长 $(P_1, P_2, P_3, P_4, P_5) = (10, 1, 2, 1, 5)$ ，优先级 $(P_1, P_2, P_3, P_4, P_5) = (3, 1, 4, 5, 2)$ ，数字越小优先级越高，求优先级调度下的平均周转时间、平均等待时间、平均响应时间

解 0-1 处理 P_2 ，1-5 处理 P_5 ，6-16 处理 P_1 ，16-18 处理 P_3 ，18-19 处理 P_4

进程	等待时间	响应时间	周转时间
P_1	6	6	16
P_2	0	0	1
P_3	16	16	18
P_4	18	18	19
P_5	1	1	6

平均等待时间为 8.2，平均响应时间为 8.2，平均周转时间为 12

5.3.5 多级队列调度

先将优先级相同的进程置于同一队列，然后队列内按照其它调度算法进行调度

例 设突发时长 $(P_1, P_2, P_3, P_4, P_5) = (4, 5, 8, 7, 3)$ ，优先级 $(P_1, P_2, P_3, P_4, P_5) = (3, 2, 2, 1, 3)$ ，求优先级 RR 下的平均周转时间、平均等待时间、平均响应时间

解 0-7 处理 P_4 ，7-9 处理 P_2 ，9-11 处理 P_3 ，11-13 处理 P_2 ，13-15 处理 P_3 ，15-16 处理 P_2 ，16-20 处理 P_3 ，20-22 处理 P_1 ，22-24 处理 P_5 ，24-26 处理 P_1 ，26-27 处理 P_5

进程	等待时间	响应时间	周转时间
P_1	22	20	26
P_2	11	7	16
P_3	12	9	20
P_4	0	0	7
P_5	24	22	27

平均等待时间为 13.8，平均响应时间为 12.8，平均周转时间为 19.2

5.3.6 多级反馈队列调度

在多级队列调度的基础上允许进程在队列间迁移，可解决无限等待、饥饿等问题

5.4 线程竞争

竞争范围：包括同一进程的线程竞争和内核线程竞争

Pthread 库允许创建线程时指定竞争范围（部分操作系统可能不支持）

5.5 多处理器调度

对称多处理自调度：可以多个处理器共用就绪队列，也可以每个处理器私有队列

多核两级调度：操作系统决定任务在处理器间的分配，处理器自行决定每个核的任务

5.5.1 负载均衡问题

多处理器系统的关键是如何维持每个处理器核的负载平衡

- 推平衡：超载处理器将任务推给其它处理器
- 拉平衡：空闲处理器将任务从其它处理器拉过来

5.5.2 处理器亲和性

进程在处理器间迁移时可能导致处理器的缓存内容失效，需要更新

保持进程在同个处理器上运行称为处理器亲和性

5.6 实时 CPU 调度

软实时：保证关键进程优先于非关键进程；硬实时：保证进程在截止时间前完成

延迟组成：包括事件延迟（请求提交到得到服务的时间）、中断延迟（CPU 收到中断到中断处理程序开始的时间）、调度延迟（停止当前进程到启动另一个进程的时间）

基于优先级调度：保证实时进程处理时间小于等于截止时间，优先级最高

单调速率调度：突发周期越短，优先级越高；突发周期越长，优先级越低

最早截止时间优先调度：截止时间越早，优先级越高；截止时间越晚，优先级越低

5.7 操作系统实例

5.7.1 Linux 调度

- 抢占式；基于优先级调度；优先级范围分为实时和非实时
- 数字越小，优先级越高；实时优先级范围 0-99，非实时优先级范围 100-140

- 具有类似轮询调度算法的时间片机制

友好值与 CPU 时间：根据友好值计算进程的虚拟运行时间 `vruntime`；运行时间越短，友好值越高，则虚拟运行时间越低；虚拟运行时间越低，则优先级越高

5.7.2 Windows 调度

- 抢占式；基于优先级调度；有轮询调度机制
- 数字越大，优先级越高；共有 32 个全局优先级
- 内存管理进程优先级为 0，持续运行
- 优先级用优先级类和相对优先级值表示，二者结合映射到全局优先级

5.7.3 Solaris 调度

- 抢占式；基于优先级调度；有多级反馈队列机制
- 同优先级进程按轮询调度

5.8 调度算法评估

5.8.1 确定性模型

采用预先确定的负载评估算法性能，例如在已经确定好突发时间的条件下，利用平均等待时间评估算法，则一定是 SJF 最优

5.8.2 排队模型

将到达时间间隔和突发时间描述为概率分布，一般用指数分布

Little 公式： $n = \lambda W$ ， n 为平均队列长度， λ 为平均到达速率， W 为平均等待时间

5.8.3 仿真

仿真比确定性模型和排队模型更精确，过程为：设置参数，根据分布生成样本，并统计系统运行的统计数据，以评估调度算法

5.8.4 实际应用

将算法直接应用更加精确，但代价更大；灵活的调度程序可根据系统参数进行应变

6 进程同步

6.1 同步问题

多任务协作过程中，对共享数据的访问可能导致数据不同步

例如生产者消费者模型中，生产者使对象数量 +1，消费者使对象数量 -1，假如两者同时取得对象数量，计算后送回，会导致对象数量取决于后送回一方的数据而不是不变

6.2 临界区问题

多任务协作过程中，访问共享数据的代码段称为临界区

进程同步的问题之一就是临界区问题，要求如下：

- 互斥：临界区只允许单个进程工作
- 进步：在有进程想要进入临界区的情况下，临界区不能空闲
- 有限等待：进程提出进入临界区的请求后，其它进程进入临界区的次数有限

6.3 Peterson 解决方案

Peterson 方案的算法如下：（ i 表示当前进程， j 表示另一进程）

```
bool flag[2];
flag[i] = true;
int turn = j;
while(flag[j] && turn == j);
/* Critical Region */
flag[i] = false;
```

原理：当前进程请求进入时，若另一进程不在临界区，则 $\text{flag}[j]$ 为 false ，当前进程可跳出 while 语句进入临界区；若另一进程在临界区，则 $\text{flag}[j]$ 为 true ，当前进程需要在 while 语句处等待，直到另一进程退出临界区使得 $\text{flag}[j]$ 变为 false ；当两个进程同时请求进入时， turn 会被瞬间改写两次，但当两个进程分别执行到 while 语句时，由于 turn 值唯一确定，只有一个进程能进入临界区

缺点：现代编译器可能会重新排布高级语言语句的执行顺序，因此不再可用，但它提供了进程同步的基本思想

6.4 硬件同步支持

6.4.1 内存屏障

内存屏障机制：处理器对内存的修改会通知其它处理器，保证后续内存操作正确

6.4.2 硬件指令

将部分变量的访问或修改操作作为不可中断的指令，便于利用这些变量进行同步

Test-and-set 指令：将目标值修改为 `true`，并返回目标值的原有值；进程进入临界区之前，不断检测 **Test-and-set**，当结果为 `false` 才可进入，并将目标值修改为 `true`；当进程退出临界区时，将目标值设置为 `false`

Compare-and-swap 指令：当目标值等于期望值时，修改目标值为新的值，并返回目标值的原有值；进程进入临界区之前，不断检测 **Compare-and-swap**，当结果为 0 才可进入，并将目标值修改为 1；当进程退出临界区时，将目标值设置为 0

6.4.3 原子变量

原子变量：提供对基本数据类型的互斥操作，也是同步工具之一

6.5 互斥锁

作用：保护临界区，防止竞争；进程进入临界区时获取锁，退出临界区时释放锁

缺点：忙等待与自旋锁（获取锁不成功时反复获取锁）

6.6 信号量

6.6.1 信号量的使用

分为整型信号量和二进制信号量，可控制多种资源的访问

`wait()` 在信号量为正时使信号量自减，否则进程将自己阻塞，交还 CPU 控制权；`signal()` 使信号量自增，并唤醒可能阻塞的进程

6.6.2 信号量的实现

对信号量的操作必须是互斥的，但由于操作比较简单，可以采用自旋锁形式实现

6.7 管程

管程在 `wait()` 调用过之前，`signal()` 无效果；此外管程还可拥有多个条件变量，由进程等待某条件时使用，可处理多种同步问题

6.8 同步与活性

活性指每个进程都要在生命周期内取得进展；一种典型的活性失败是死锁，指进程之间互相等待，导致无限等待

7 同步案例

7.1 经典同步问题

7.1.1 有界缓冲区问题

生产者消费者模型中，mutex 提供缓冲区互斥

empty 和 full 分别表示有空闲的缓冲区/缓冲区中有消息

生产者进程：

```
while(true)
{
    /* Produce Object */
    wait(empty);
    wait(mutex);
    /* Put object into buffer */
    signal(mutex);
    signal(full);
}
```

消费者进程：

```
while(true)
{
    /* Produce Object */
    wait(full);
    wait(mutex);
    /* Put object into buffer */
    signal(mutex);
    signal(empty);
}
```

7.1.2 读者作者问题

模型：读者只会读取数据，作者可读取也可修改数据

因此临界区内可以有多个读者，但只能有单个作者

rw_mutex 提供临界区互斥，rd_cnt 跟踪读者数量，mutex 提供对 rd_cnt 访问的互斥

作者进程：

```
while(true)
{
    wait(rw_mutex);
```

```
    /* Perform operation */  
    signal(rw_mutex);  
}
```

读者进程:

```
while(true)  
{  
    wait(mutex);  
    rd_cnt++;  
    // Wait for rw as the first reader  
    if (rd_cnt==1) wait(rw_mutex);  
    signal(mutex);  
    /* Perform operation */  
    wait(mutex);  
    rd_cnt--;  
    // Release rw as the last reader  
    if (rd_cnt==0) signal(rw_mutex);  
    signal(mutex);  
}
```

上述代码未解决读者进程持续进入导致作者进程饥饿的问题，实际设计时需要考虑

7.1.3 哲学家就餐问题

模型： n 个哲学家围坐在圆桌旁，每相邻 2 个哲学家之间有 1 根筷子，哲学家只有拿到 2 根筷子才能就餐，吃完饭后放下筷子

信号量解决方案:

```
// Shared semaphore: c[n]  
// For philosopher[i]:  
while (true)  
{  
    wait(c[i]);  
    wait(c[i+1]);  
    signal(c[i]);  
    signal(c[i+1]);  
}
```

信号量方案的问题：每个哲学家都拿起左边的筷子会导致无限等待

管程解决方案:

```
// Philosopher states: state[5], T-thinking, H-hungry, E-eating
// Monitor: self[5] for wait and signal
// For Philosopher[i]:
void test(int i) {
    if (state[(i+1)%5] != E && state[(i+4)%5] != E && state[i] == H) {
        state[i] = E;
        self[i].signal();
    }
}
void pickup(int i) {
    state[i] = H;
    test(i);
    if (state[i] != E) self[i].wait();
}
void putdown(int i) {
    state[i] = T;
    test((i+1)%5);
    test((i+4)%5);
}
```

管程方案的问题：避免了死锁，但可能出现饥饿

7.2 内核的同步

7.2.1 Windows 同步

提供中断屏蔽（单核系统）、自旋锁（多核系统）、互斥锁、信号量、事件、定时器等机制，使用自旋锁的进程不会被抢占

7.2.2 Linux 同步

提供信号量、原子操作、自旋锁等同步工具，还提供了读者作者模型的解决方案等

7.3 POSIX 同步

7.3.1 POSIX 互斥锁

用 `pthread_mutex_t` 声明，`pthread_mutex_init` 初始化锁，`pthread_mutex_lock` 获取锁，`pthread_mutex_unlock` 释放锁

7.3.2 POSIX 信号量

命名信号量由 `sem_t*` 声明，可在不同进程的线程间共享
无名信号量由 `sem_t` 声明，只能在同一进程的线程间共享
`sem_wait` 获取锁，`sem_post` 释放锁

7.3.3 POSIX 条件变量

由于 C 语言不提供管程，所以 POSIX 条件变量用互斥锁保证互斥

7.4 Java 的同步

7.4.1 Java 管程

每个类都有自己的锁，当 Java 方法声明为 `synchronized` 时，调用方法的线程必须拥有锁才能进入；若锁已被占用，则进入入口集等待；方法退出时，会自动归还锁，并令入口集的某个线程进入临界区；入口集的线程可进行竞争

7.4.2 Java 重入锁

重入锁保证临界区异常时能够释放锁，用法例如（`key` 是重入锁）：
`key.lock(); try{/* Critical Region */} finally {key.unlock()};`

7.4.3 Java 信号量

Java 信号量的初值可以为负；`acquire()` 方法在信号量为正时使其自减，在信号量为负时通过抛出异常的方式中断；`release()` 方法使信号量自增

7.4.4 Java 条件变量

条件变量的创建与重入锁相关联，需调用重入锁的 `newCondition()` 方法

7.5 其他方法

7.5.1 事务内存

在函数中插入 `atomic{} 语句块` 保证操作原子执行

7.5.2 OpenMP

在函数中插入编译指令 `#pragma omp critical{} 语句块` 保证操作原子执行

8 死锁

8.1 系统模型

计算机系统可视为由多种资源组成，如 CPU 处理周期、内存、输入输出设备等资源的使用由申请、使用、释放组成

8.2 死锁的出现

例如，创建了 2 个互斥锁 `mutex1` 和 `mutex2`

线程 `T1` 依次调用 `mutex1.lock()` 和 `mutex2.lock()`

线程 `T2` 依次调用 `mutex2.lock()` 和 `mutex1.lock()`

假如 `T1` 锁定 `mutex1` 而 `T2` 锁定 `mutex2`，就会导致无限等待成为死锁

8.3 死锁特点

8.3.1 必要条件

- 互斥：同时只有单个线程能够使用资源
- 占有并等待：占有资源的同时等待其他资源
- 非抢占：资源只能由线程资源释放
- 循环等待：等待关系形成有向闭链

8.3.2 资源分配图

点集 $V = T \cup R$ ， T 为线程集， R 为资源集

$T_i \rightarrow R_j$ 表示申请边， $R_m \rightarrow T_n$ 表示分配边

资源分配图存在有向闭链时可能存在死锁，不存在有向闭链时一定不存在死锁

8.4 死锁处理策略

- 死锁预防策略保证不会出现死锁
- 通过协议避免死锁，并认为系统不会发生死锁
- 允许进入死锁状态，检测到死锁后进行修复

8.5 死锁预防

8.5.1 取消互斥

对于多个以只读方式访问某资源的线程，可以取消互斥

8.5.2 禁止占有并等待

线程申请某资源失败时应将所有已申请资源释放

缺点：资源利用率较低；使用资源较多的线程会饥饿

8.5.3 允许抢占

正在等待的线程已申请的资源可被其它线程抢占

不能用于互斥锁和信号量等资源

8.5.4 禁止循环

为每种资源分配序号，每个线程只能按照序号递增的顺序申请资源

8.6 死锁避免

系统通过额外信息判断是否可能发生死锁，进而令部分线程挂起以避免死锁

8.6.1 安全状态

若存在线程序列 $\langle P_1, P_2, P_3, \dots, P_n \rangle$ 满足： $\forall P_i$ ，剩余资源和 $\forall P_j (j < i)$ 占有的资源之和能够满足使得 P_i 运行所需的最大资源，称为安全状态

8.6.2 资源分配图算法

线程申请资源时，在资源分配图中检测分配后是否包含有向闭链，若有则不允许申请

8.6.3 银行家算法

设有 n 个线程共享 m 种资源

$available[m]$ 表示当前每种资源的可用数量

$max[n][m]$ 表示每个线程运行需要的全部资源

$allocated[n][m]$ 表示已为每个线程分配的资源

$need[n][m]$ 表示每个线程能够运行仍然需要的资源

满足： $max=allocated+need$ ，所以无需单独存储 $need$

安全性验证算法：

1. 初始化 $tmp[1..m]=available[1..m]$ ； $finish[1..n]=false$
2. 寻找满足 $finish[i]=false$ ； $need[i][1..m] \leq tmp[1..m]$ 的线程 i ，若找到则转第 3 步，若未找到则转第 4 步
3. 更新 $tmp[1..m]=tmp[1..m]+allocated[i][1..m]$ ； $finish[i]=true$ ，转第 2 步

4. 若 $\text{finish}[1..n]$ 均为 true ，则系统处于安全状态

资源请求算法：假设 $R[1..m]$ 表示线程 i 申请资源的向量

1. 若 $R[1..m] \leq \text{need}[i][1..m]$ ，转第 2 步，否则拒绝申请
因为该线程此时申请的资源已经超过了事先声明的最多资源
2. 若 $R[1..m] \leq \text{available}[1..m]$ ，转第 3 步，否则拒绝申请
因为此时的可用资源不能满足该申请
3. $\text{available}[1..m] = \text{available}[1..m] - R[1..m]$
 $\text{allocated}[i][1..m] = \text{allocated}[i][1..m] + R[1..m]$
并运行安全检测算法，若仍处于安全状态则接受申请，否则拒绝申请并恢复原状

例 银行家算法的数据结构如下面几个表所示：

allocated				max				available			
P_1	0	1	0	P_1	7	5	3				P_2
P_2	2	0	0	P_2	3	2	2	P_3	9	0	2
P_3	3	0	2	P_3	9	0	2	P_4	2	2	2
P_4	2	1	1	P_4	2	2	2	P_5	4	3	3
P_5	0	0	2	P_5	4	3	3				

1. 说明系统处于安全状态
2. 申请序列 $R_2 = (1, 0, 2)$, $R_5 = (3, 3, 0)$, $R_1 = (0, 2, 0)$ 依次到达，接受安全的申请并更新状态，拒绝不安全的申请并说明理由

解 首先列出 need 表

need			
P_1	7	4	3
P_2	1	2	2
P_3	6	0	0
P_4	0	1	1
P_5	4	3	1

1. P_2 可运行，此后 $\text{available} = (5, 3, 2)$
 P_4 可运行，此后 $\text{available} = (7, 4, 3)$
此时满足任意线程的运行，所以安全
2. $R_2 = (1, 0, 2)$ 由安全检测，只要 P_2 申请的资源不超过 $\text{max}[2]$ 即是安全的，更新状态为

allocated				need						
P_1	0	1	0	P_1	7	4	3			
P_2	3	0	2	P_2	0	2	0	available		
P_3	3	0	2	P_3	6	0	0	2 3 0		
P_4	2	1	1	P_4	0	1	1			
P_5	0	0	2	P_5	4	3	1			

$R_5 = (3, 3, 0)$ 拒绝, 因为当前可用资源不能满足申请

$R_1 = (0, 2, 0)$ 拒绝, 因为分配后将不存在可运行的线程, 导致死锁

8.7 死锁检测的实际应用

8.7.1 每种资源只有单个实例

将资源分配图中的进程 $\rightarrow$ 资源 $\rightarrow$ 进程路径合并为进程 $\rightarrow$ 进程, 从而得到等待图, 并通过检测等待图中的回路检测死锁, 时间复杂度 $O(n^2)$

8.7.2 每种资源存在多个实例

可应用银行家算法, 时间复杂度 $O(mn^2)$

8.7.3 检测算法的使用

死锁检测的使用频率取决于死锁出现的频率以及死锁影响的线程数量
可在 CPU 使用率较低时检测是否出现了死锁

8.8 死锁恢复

可释放所有线程令其重新执行, 或单次释放单个线程并逐次检测死锁直到死锁消失
本质上也是一种资源抢占, 需要考虑线程回滚、优先级、饥饿等问题

9 内存

9.1 背景

9.1.1 硬件

程序只有加载到内存中才能运行，内存、高速缓存、寄存器是 CPU 可直接访问的存储单元的视角就是一系列操作，操作的格式包括地址+读请求、地址+数据+写请求
为保护其它进程，进程的执行过程中只能在自己的地址空间内寻址

9.1.2 地址绑定

程序可能位于内存任意位置，因此程序中的地址要和存储器地址相符，称为地址绑定
编译时绑定生成的代码称为绝对代码，载入内存的地址改变时必须重新编译
加载时绑定生成的代码称为可重定位代码，在内存中迁移时需要重新载入内存
执行时绑定生成的代码能够以较小开销在内存中迁移，是当前最多采用的方法

9.1.3 虚拟地址与物理地址

虚拟地址是由 CPU 生成的地址，物理地址是内存能够识别的地址

编译时绑定和加载时绑定生成的代码中物理地址与虚拟地址相同，执行时绑定生成的代码中物理地址与虚拟地址不同

需要额外的内存管理单元进行物理地址和虚拟地址之间的映射

9.1.4 动态加载与动态链接

动态加载：不需要将整个程序载入内存，而只加载正在调用的程序段

动态链接：编译过程不会链接，执行时才会链接；链接库可由多程序共享

9.2 连续内存管理

目的：操作系统与应用程序共享内存，每个进程应处于连续的空间内

9.2.1 动态视角

内存可视为连续空间，新进程创建时占用空间，退出时释放空间；运行一段时间后，内存将成为孔状结构，可用空间与进程占用空间交替分布

为进程分配内存的算法：首次适应（分配首个足够大的可用连续空间）；最优适应（分配足够大的连续可用空间中最小的）；最差适应（分配足够大的连续可用空间中最大的）

9.2.2 碎片问题

外部碎片问题指全部碎片的空间之和较大，能够满足某进程需求，但不连续，无法使用；内部碎片问题指为进程分配的内存大于进程所需的内存，造成内部空间浪费

对首次适应的分析表明，稳定时约 50% 的内存浪费于碎片

可移动进程的内存块以将其合并，得到更大的可用空间

9.3 分页内存管理

9.3.1 基本方法

进程的内存可以不连续，可分为等长的块加以管理，减少外部碎片问题

内存块称为帧或页，物理内存的块一般称为物理帧，虚拟内存的块一般称为虚拟页

建立页表维护物理地址和虚拟地址的映射，将 CPU 生成的虚拟地址分为两部分

- 页码：在页表中转化为物理帧的基地址
- 页内偏移：与物理帧的基地址结合，用于实际寻址

由于地址是二进制表示的，所以页的大小为 2^k 字节，目前一般为 4KB~1GB

当虚拟地址空间为 2^m ，页的大小为 2^n 时，前 $m - n$ 位为页码，后 n 位为页内偏移

内部碎片的形式：不足单页大小的实际内容占用整页；当页的大小减小时，内部碎片会减小，但页的数量会增多，导致访问时间变长

9.3.2 硬件支持

最初，页表是保存在内存中的，每取 1 次地址要访问 2 次内存

硬件优化：转换后援缓冲器 (TLB) 作为独立的硬件，可支持访问频率较高的页表查找

9.3.3 保护

为防止进程访问其它进程的内存，需要设置保护位表示帧的操作权限

可在每个在页表项中附加有效位表示当前进程是否有权访问/修改此页的内容

9.3.4 共享页

可通过不同虚拟页指向相同物理帧的方式，实现内存共享和进程间通信

9.4 页表结构

9.4.1 多级页表

多级页表虚拟地址由二级页码、页码、页内偏移构成；在二级页表中二级页码转换为页表的基地址；在页表中以页码为偏移量查找到页的基地址；最后在页内根据页内偏移寻址

9.4.2 哈希页表

根据虚拟地址的页码值，在哈希表中匹配到物理地址，利用哈希方法加快访问速度

9.4.3 倒置页表

倒置页表按照物理帧的顺序排序，根据进程号和虚拟地址转换得到物理地址；标准页表可将多个虚拟地址映射到相同物理地址，倒置页表只允许每个物理帧对应唯一的虚拟页

9.4.4 TLB 的组织

CPU 需要寻址时，先去 TLB 查找；若找到可直接转换地址；否则需要查找内存中的页表，并创建转换条目到 TLB，便于多次重复访问少数内存页

9.5 交换

交换是指将暂时不需要的进程换出内存到外存的交换区，将需要的进程从外存的交换区换入到内存；调页允许全部进程的物理空间之和大于内存物理空间

快速外存：访问速度慢于内存但快于一般的外存，容量大于内存，可用作交换区

分页交换：只交换进程的部分页面，可与分页内存管理和虚拟内存机制相配合

移动操作系统的闪存容量较小且吞吐量差，一般不支持交换，需要其他方式释放内存

9.6 实际应用

9.6.1 x86 架构

逻辑地址为 48 位，前 16 位为选择器，后 32 位用于分页寻址

页地址扩展采用 3 级分页将地址空间增加到 36 位，支持 64GB 寻址

9.6.2 x64 架构

采用四级页表，将 48 位虚拟内存转换为 52 位物理内存

9.6.3 ARM-v8 架构

采用 2 级 TLB 的方式，分为 2 个微 TLB 和 1 个主 TLB，加速地址转换与内存访问

10 虚拟内存

10.1 背景

程序由于有不常用的功能，执行时不必全部位于内存中；程序只有部分处于内存中，有利于允许更多应用程序执行，提高 CPU 利用率和吞吐量，减少周转时间和响应时间

虚拟内存的作用之一就是让程序在部分位于内存中的情况下仍然可以视自己为整体，是进程在内存中的逻辑映像，还可与分页内存管理进行配合等

10.2 请求调页

10.2.1 基本概念

仅在进程需要时才将特定页面调入内存，可减少内存与外存的交换，还可进行保护等
当进程访问某个地址时，先检查对应页是否位于内存中；若否，则触发缺页错误
操作系统处理缺页错误的过程（缺页错误也是中断，以下仅是中断处理程序）：

1. 检查页表，确定访问的合法性（只能访问本进程的页和共享的页）
2. 若不合法，阻止操作；若合法，寻找物理内存中的空闲帧，移入对应页面
3. 修改页表的相关内容，并重启进程的触发缺页错误的指令

若进程初始化时所有页都未调入内存，首条指令必然触发缺页错误，称为纯请求调页

分析表明，进程频繁访问的页面占进程所有页面的比例很小，意味着进程经常活动的范围有限；计算机程序的以上性质称为局部性，虚拟内存和分页管理正是利用了局部性原理，更有利于硬件资源的充分利用

10.2.2 空闲帧列表

空闲帧通常用链表存储，保存空闲帧基址和空闲帧大小

10.2.3 请求调页的性能

设缺页错误率为 p ($0 \leq p \leq 1$)，平均访问时间为 T ，内存访问时间为 t_{mem} ，缺页错误处理时间为 t_{pf} ，则有

$$T = (1 - p) \times t_{mem} + p \times t_{pf}$$

优化：将交换内容从文件系统中抽象出来；进程启动时直接将内容移到交换区；内存中的只读内容无需换出内存，可直接覆盖，需要该只读内容时再从文件系统读入

10.3 写时复制

- `fork()` 系统调用：父进程和子进程共享内存帧
当且仅当任意进程进行修改内存操作时才复制内存帧
- `vfork()` 系统调用：子进程执行时挂起父进程
父进程恢复时，对子进程的修改可见

10.4 页面置换算法

10.4.1 基本页面置换

根据修改状态优化页面置换：交换区保存所有页，内存帧需要换置换时先检查是否修改过，未修改过的内存帧无需写出到交换区

页面置换算法：目的是减少内存访问过程中缺页错误的次数；可通过模拟内存访问的页号序列，统计缺页错误的次数，对页面置换算法进行评估

帧分配算法：根据进程执行的任务，决定为进程分配的帧数

缺页错误处理过程：

1. 找到所需页的物理位置
2. 寻找空闲帧，若找到空闲帧，则使用空闲帧；若未找到则选择牺牲帧，根据牺牲帧是否修改过，选择写回或者不写回外存，同时修改页表
3. 将所需页读入空闲帧，修改页表
4. 重新执行发生缺页错误的指令

10.4.2 FIFO 页面置换

最先进入内存的页面先换出内存，问题：添加内存帧时可能导致更多的缺页错误

10.4.3 LRU 页面置换

最长时间未使用的页面先换出内存（看过去），一般认为是可实现的最优算法

10.4.4 OPT 页面置换

最后将要被使用的页面先换出内存（看未来），难以预测不能实现，但可作为评判标准

例 有内存引用的页号序列 7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1，内存可用 3 帧，分别求 FIFO LRU OPT 下的缺页次数

解 内存帧的情况如下表所示（其中 LRU 的下标为该帧未使用的次数）

统计缺页错误的次数可以得知 FIFO LRU OPT 的缺页次数分别为 15 12 9

所以 OPT 为最优算法，LRU 次之，FIFO 性能最差

引用	FIFO			LRU			OPT		
7	7	∅	∅	7	∅	∅	7	∅	∅
0	7	0	∅	7 ₁	0	∅	7	0	∅
1	7	0	1	7 ₂	0 ₁	1	7	0	1
2	2	0	1	2	0 ₂	1 ₁	2	0	1
0	2	0	1	2 ₁	0	1 ₂	2	0	1
3	2	3	1	2 ₂	0 ₁	3	2	0	3
0	2	3	0	2 ₃	0	3 ₁	2	0	3
4	4	3	0	4	0 ₁	3 ₂	2	4	3
2	4	2	0	4 ₁	0 ₂	2	2	4	3
3	4	2	3	4 ₂	3	2 ₁	2	4	3
0	0	2	3	0	3 ₁	2 ₂	2	0	3
3	0	2	3	0 ₁	3	2 ₃	2	0	3
0	0	2	3	0	3 ₁	2 ₄	2	0	3
3	0	2	3	0 ₁	3	2 ₅	2	0	3
2	0	2	3	0 ₂	3 ₁	2	2	0	3
1	0	1	3	1	3 ₂	2 ₁	2	0	1
2	0	1	2	1 ₁	3 ₃	2	2	0	1
0	0	1	2	1 ₂	0	2 ₁	2	0	1
1	0	1	2	1	0 ₁	2 ₂	2	0	1
7	7	1	2	1 ₁	0 ₂	7	7	0	1
0	7	0	2	1 ₂	0	7 ₁	7	0	1
1	7	0	1	1	0 ₁	7 ₂	7	0	1

10.4.5 LRU 算法的实现与近似

堆栈：引用页面时，若未在栈中则将其入栈否则移到栈顶；页面置换时牺牲栈底页面

计时器实现：引用页面时写入引用时刻的时钟计数，页面置换时牺牲时钟最早的页面

额外引用字节近似：为每个页面保留额外引用字节；引用内存页时将引用位设为 1；操作系统定期将引用位放置在引用字节最高位，丢弃引用字节最低位，并将引用位重置，近似最近使用情况；页面置换时选择引用字节最小的页面作为牺牲帧

第二次机会近似：检查引用位，若为 1 则置 0 并检查下个页面，若为 0 则选为牺牲帧

增强第二次机会近似：在引用位基础上添加修改位，结合使用情况和修改情况选择牺牲帧；系统空闲时可将修改过的页面进行写出并重置修改位

10.5 帧分配

10.5.1 帧的最小值与最大值

帧的最小值：保证单条指令能执行的最小帧数，防止陷入缺页错误与指令重启的循环

帧的最大值：即物理内存中的可用帧数

分配算法为进程分配的帧应大于帧的最小值，为所有进程分配的帧应小于总物理帧数

10.5.2 全局分配与局部分配

全局分配：进程获得的帧数是动态的，可从其他进程获取帧

局部分配：进程获得的帧数在进程运行期间保持不变，只能利用分配给本进程的帧

收割者策略：可用内存量小于阈值时触发收割，从进程处回收页面

10.5.3 非均匀内存访问

在 NUMA 架构中，每个 CPU 有自己的快速内存，而对于其他 CPU 而言是慢速内存
此时进程的帧应当分配在其线程所在的 CPU 的快速内存上以加速访问

10.6 抖动

10.6.1 抖动现象

当为进程分配的帧过少时，缺页错误触发率很高，导致存储系统在不停地换入换出；不仅会降低 CPU 利用率，还可能让操作系统认为缺页错误率高，需要提高并发度导致更多缺页错误，使 CPU 利用率进一步降低

10.6.2 抖动预防

工作集：最近一定数量的页面引用中的页面，若总工作集大于物理帧可能会出现抖动

缺页错误率：可作为比工作集模型更为直接的抖动预防依据

10.7 内存压缩

可用物理内存较少或不足时，将多个内存帧压缩为单个内存帧，引用被压缩的内存帧需要解压，也可减少页面交换次数

10.8 虚拟内存性能的其他因素

预调页面：减少进程启动时发生的大量页面错误

页面大小：现代操作系统的页面大小通常为 $2^{12} \sim 2^{22}$ 字节，随硬件加速而逐渐增大

TLB 大小：TLB 增大时命中率将提高，但 TLB 本身的访问也将变慢

程序结构：执行顺序较为接近的指令使用较为接近的内存空间时缺页错误较少

IO 联锁：将 IO 固定在内存的特定空间以加速内存和外存交换

10.9 操作系统实例

10.9.1 Linux

slab 由一个或多个物理上连续的页面组成，状态有半满、全满和全空三种

Linux 使用 slab 分配，优先使用半满的结构，再使用全空的结构

10.9.2 Windows

采用集群方式请求调页，缺页错误调入请求页面和其前后的页面

在可用内存量低于阈值时启动收割者，将内存可用量恢复至阈值以上

10.9.3 Solaris

维护空闲页顺序表为缺页错误的线程分配页面，并为页面提供优先级分划

11 大容量存储

11.1 大容量存储结构

11.1.1 硬盘驱动器 HDD

HDD 的参数包括每分钟转数、传输速率、随机访问时间，访问延迟=寻道时间+旋转延迟，平均 IO 时间=平均访问延迟+控制器延迟

11.1.2 非易失性存储设备 NVM

NVM 包括固态硬盘、USB 驱动器和 DRAM 棒等，数据不易丢失且访问速度较快，但单位容量成本更高、单设备容量较小，且访问速度可能受到串口限制

一种常用的 NVM 是 NAND，其数据不能覆盖只能擦除后重写，擦除次数过多会导致损坏，因此可用写入次数衡量寿命；同时 NAND 控制器要保证每部分擦除写入次数相近

部分计算机为 NVM 提供了 NVMe 以提高吞吐和减少延迟

部分易失性存储器例如 DRAM 也可作为较为快速的二级存储，用于交换区等

11.1.3 地址映射

可将大容量存储视为逻辑块的数组，逻辑块即物理扇区的映射

11.2 HDD 调度

磁盘调度目标是减少访问时间、提高访问速率，HDD 访问时间可用磁头移动距离近似 IO 请求由操作系统或用户程序发出，信息包括输入输出类型、内存地址、外存地址、数据量等，操作系统维护请求队列，在外存繁忙时启用调度

磁盘调度过去由操作系统完成，现在由磁盘控制器完成

11.2.1 FCFS 调度

按照磁盘请求序列依次移动到目标位置处理

11.2.2 SCAN 调度

磁头不断移动，处理中途有请求的位置，到达端点时反向移动

11.2.3 C-SCAN 调度

磁头不断移动处理中途有请求的位置，到达端点时反向移动且不处理任何请求，直到到达起点再反向扫描并处理请求

例 磁盘范围 0~199，磁头开始位于 53 处，访问请求序列 98 183 37 122 14 124 65 67，求 FCFS SCAN C-SCAN 调度下磁盘总移动距离

解 FCFS: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67 总距离 640

SCAN: 53 → 0 → 183 总距离 236

C-SCAN: 53 → 199 → 0 → 37 总距离 382

11.2.4 算法选择

请求数量较少时各种算法均类似于 FCFS，请求较多时算法应设置防止饥饿的机制

11.3 NVM 调度

NVM 更适合随机访问，IOPS 更大，可直接用 FCFS 和合并相邻请求的调度方式
缺点是随写入次数增加，NVM 写入的速度会下降，称为写放大效应

11.4 错误检测和纠正

校验和：利用模运算检测错误，例如奇偶校验

循环冗余校验：使用函数检测错误，能检测多个位的错误

11.5 存储设备管理

物理格式化：将磁盘分为控制器可读写的块，头部或尾部包含使用信息等

逻辑格式化：操作系统将磁盘划分为若干个块，每个块视为逻辑盘，并创建文件系统

引导程序：位于外存的固定位置，启动计算机时首先运行，将内核载入内存

原始磁盘/原始分区：不使用文件系统而作为数组，可用于向数据库提供存储服务等

11.6 交换空间管理

尽可能大，以支持更大的虚拟内存；尽可能分散于多设备，以减少单设备 IO；尽可能有专用交换空间；可以位于原始分区也可以位于文件系统

11.7 存储连接方式

主机连接：计算机通过串口和 IO 总线访问存储设备的方式

网络连接：通过网络提供远程连接，通过远程调用实现主机与存储的连接

云存储：通过互联网实现共享的数据中心，基于 API 提供服务

存储区域网：多主机连接多存储阵列，每个主机可有私有存储，也有多主机共享存储

12 IO 系统

12.1 概述

设备驱动程序：为外部设备提供统一的访问接口，包括 IO 设备

12.2 硬件

IO 硬件包括存储设备、传输设备、人机交互设备等

接口：设备与计算机通信渠道；总线：数据传输的通路

控制器：管理端口总线设备，包含处理器、微代码、缓存

12.2.1 内存映射 IO

将设备寄存器的空间映射为内存空间，提供除运算指令外的 IO 操作；IO 可直接与内存交流，提高大规模数据传输效率

12.2.2 中断

CPU 设有中断请求线路 INTR，每条指令执行后检查，检测到中断后转到中断处理程序处理中断；CPU 也可设置为不允许中断的状态，保证重要过程连续

中断向量表：将中断类型与对应的中断处理程序联系的查找表，可包含优先级系统（高优先级中断可中断低优先级中断处理程序）和中断链（单个中断号对应多个中断处理程序）

12.2.3 直接内存访问 DMA

专门处理内外存 IO 的硬件，传输过程中会偷取 CPU 周期，但依然高效，过程如下：

1. CPU 通知驱动程序传输数据
2. 驱动程序通知硬件控制器传输数据
3. 硬件控制器准备好数据后发起 DMA 传输
4. DMA 传输数据，完成后通过中断通知 CPU 传输完成

12.3 应用程序的 IO 接口

操作系统内核隐藏了不同设备的差异，将 IO 设备抽象为通用类，设计通用驱动程序包括块与字符设备、网络设备、时钟与定时器、非阻塞与异步 IO、向量 IO 等设备

12.4 内核 IO 子系统

包括 IO 调度、缓冲区、缓存（内容的副本）、假脱机（处理低速输出）、设备预留（提供互斥操作）、错误处理、保护、内核数据结构、能耗管理等内容

12.5 IO 请求与设备操作转换

操作系统首先确定含有文件的设备，将文件名转为机器码，从磁盘读取到控制器缓存，标识为数据可用，并将控制权交还进程

12.6 流

流是设备驱动程序与用户进程的全双工连接，结构包括流头、驱动端、流队列

12.7 性能提高

可用减少上下文切换、数据复制、中断，使用 DMA 和设备优化，平衡 CPU、内存、总线、IO 的性能等方式提高 IO 的性能

13 文件系统接口

13.1 文件

文件本质上是连续的逻辑地址空间，包括数据文件（数字、字符、文本、二进制等格式）和可执行文件（源文件、程序文件等）

13.1.1 文件属性

文件的属性包括名称（方便人的使用）、标识符（唯一标记）、类型（支持不同类型文件的系统使用）、位置（指向设备及文件的物理位置）、大小、保护信息、时间、用户标识、目录以及其他的扩展属性

13.1.2 文件操作

文件可视为抽象的数据结构，基本操作包括创建、访问、读写、删除、清空、保存等操作系统维护文件的访问信息、访问权限，并提供互斥机制等

13.1.3 文件类型

操作系统可利用文件类型确定解读内容的方式，常用“文件名+扩展名”的形式

13.1.4 文件结构

文件结构包括字符串字节串等无格式、文本行等简单格式，也可以有复杂结构

13.2 访问方法

基本访问方法包括顺序访问和随机访问，也有复杂访问方法例如文件索引等

13.3 目录

目录是磁盘中包含文件信息的结点，便于快速定位文件，也可为文件命名便于使用

13.4 保护

操作系统可为用户或用户组添加文件权限，以限制不同用户访问

14 文件系统实现

14.1 文件系统结构

文件系统的基本结构包括逻辑存储单元和文件相关信息

设计目标是便于用户使用、提供高效的地址映射和访问、便于数据的存储定位提取

14.2 文件系统操作

卷包含引导控制块（若包含操作系统）、卷控制块、目录结构、文件控制块、安装表

14.3 目录实现

可用线性表、查找树、哈希表等方式实现文件系统的目录

14.4 分配方法

连续分配：每个文件在存储设备上占有连续的块

链接分配：文件在设备上占据不连续的块，每个块包含指向下个块的指针

索引分配：支持复杂结构的、多级的、树状的链接分配

inode 组织形式：文件内容部分依次包括直接块、一级间接块指针、二级间接块指针、三级间接块指针、……，一级间接块指针指向直接块，二级间接块指针指向由一级间接块指针组成的块数组，依此类推

14.5 空闲空间管理

位图：用比特数组保存各块状态，例如为 1 则对应块空闲，为 0 则对应块占用

链表：每个块包含下个空闲块的指针，一般不需要遍历全表

组合：支持树状结构和多级的空闲块存储

计数：存储连续空闲块的起始块号和连续个数，可用平衡查找树辅助使用

修整未使用的块：HDD 在释放块后未使用的块会保留（不进行擦除操作），再使用时会重写；NVM 会组合已释放的块进行擦除

15 文件系统内部细节

15.1 存储系统

通用计算机可包含多种存储；设备可分区，区组合成卷，卷包含文件系统

15.2 文件系统挂载

文件系统必须挂载才能使用，挂载之后可通过挂载点访问文件系统

15.3 分区与挂载

只包含块未挂载文件系统的区域称为原始分区，挂载了文件系统的区域称为卷

引导块包含引导程序（将操作系统内核从文件系统加载入内存），部分引导块还有引导控制程序（允许用户从多个操作系统中选择）；包含引导块的分区称为根分区，启动时挂载；其他分区可启动过程中自动挂载，或手动挂载

15.4 文件共享

允许多用户/多系统共享相同文件，需要考虑保护、互斥等机制

15.5 虚拟文件系统

隐藏文件系统操作的实现细节，提供通用的、面向对象方法的系统调用

15.6 远程文件系统

远程文件系统包括服务器客户机模型和分布式文件系统，可提供远程服务

15.7 一致性语义

一致性问题类似于同步问题，即多个用户如何访问共享文件、修改文件如何通知全局
Unix 语义：每个文件只有单个映像，修改立即全局可见，多用户通过同个指针访问
会话语义：修改只对修改后打开文件的用户可见，每个文件有多个物理映像